

社区开发者共建实践

Jiuwenswarm&CANNBot 让小艺辅助Ascend C开发

古法编程到一句话实现的进化之旅

徐洋帆 | CSDN「工具人呵呵」 | 上海 CANN Meetup



 我这一年：从手搓算子到指挥智能体

 CANNBot来了：一句话召唤蜂群

 实操部署：2小时从0到1

 开发者视角总结

 古法编程：那些年踩过的坑

 Agent分工：从一人多角到各司其职

 小艺加持：手机上写算子

作为开发者，我关心的是：这东西能不能让我少踩坑？

开源运行时

openJiuwenSwarm

以前一个人扛全流程，现在一句话召唤蜂群
各Agent各司其职，Skill越用越强

手机端分发

小艺开放平台

地铁上用语音让小艺查API
不用开电脑就能写算子

领域技能库

CANNBot

每个Skill对应我踩过坑的一个坑：
编译不过、API翻半天、精度对不上、性能调不动...

开发者体感从开源运行时到手机端，动动嘴就能写算子、查API -- 这才是我想要的开发体验

对比维度	openJiuwenSwarm	小艺开放平台	CANNBot
开发者体感	一句话召唤蜂群	手机上写算子	踩坑方案库
生态依托	开源社区	HarmonyOS	CANN / 昇腾
关键产出	多Agent协作+Skill自演进	智能体接入	30 Skill + 8 Agent

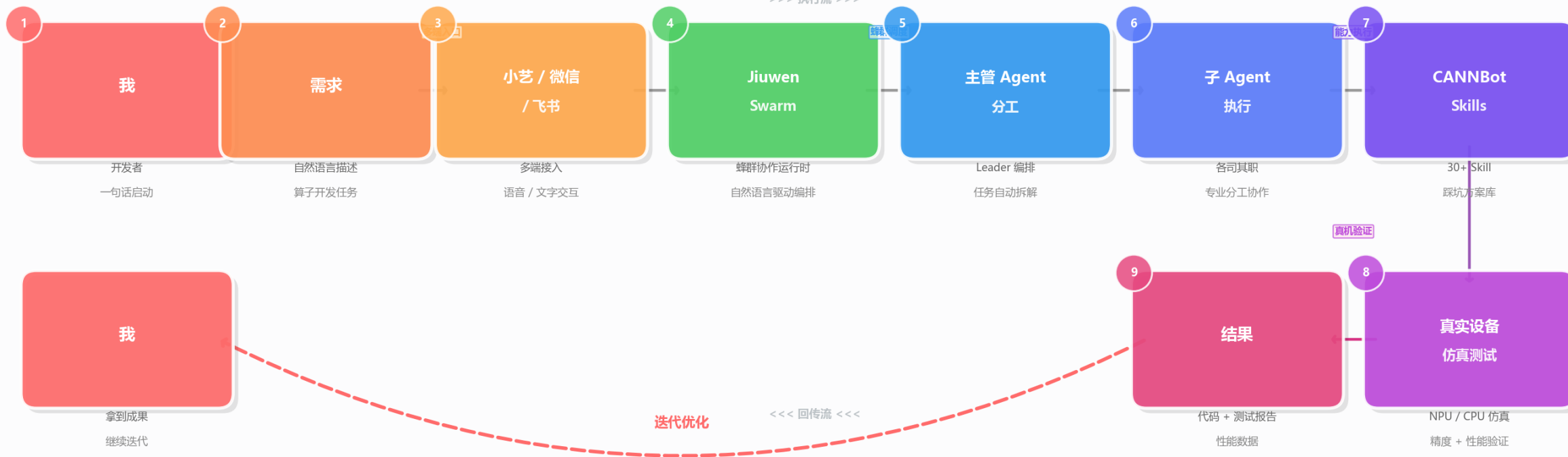
如果你也在踩Ascend C开发的坑，这些工具值得试试



一句话召唤蜂群：Ascend C 算子开发全流程

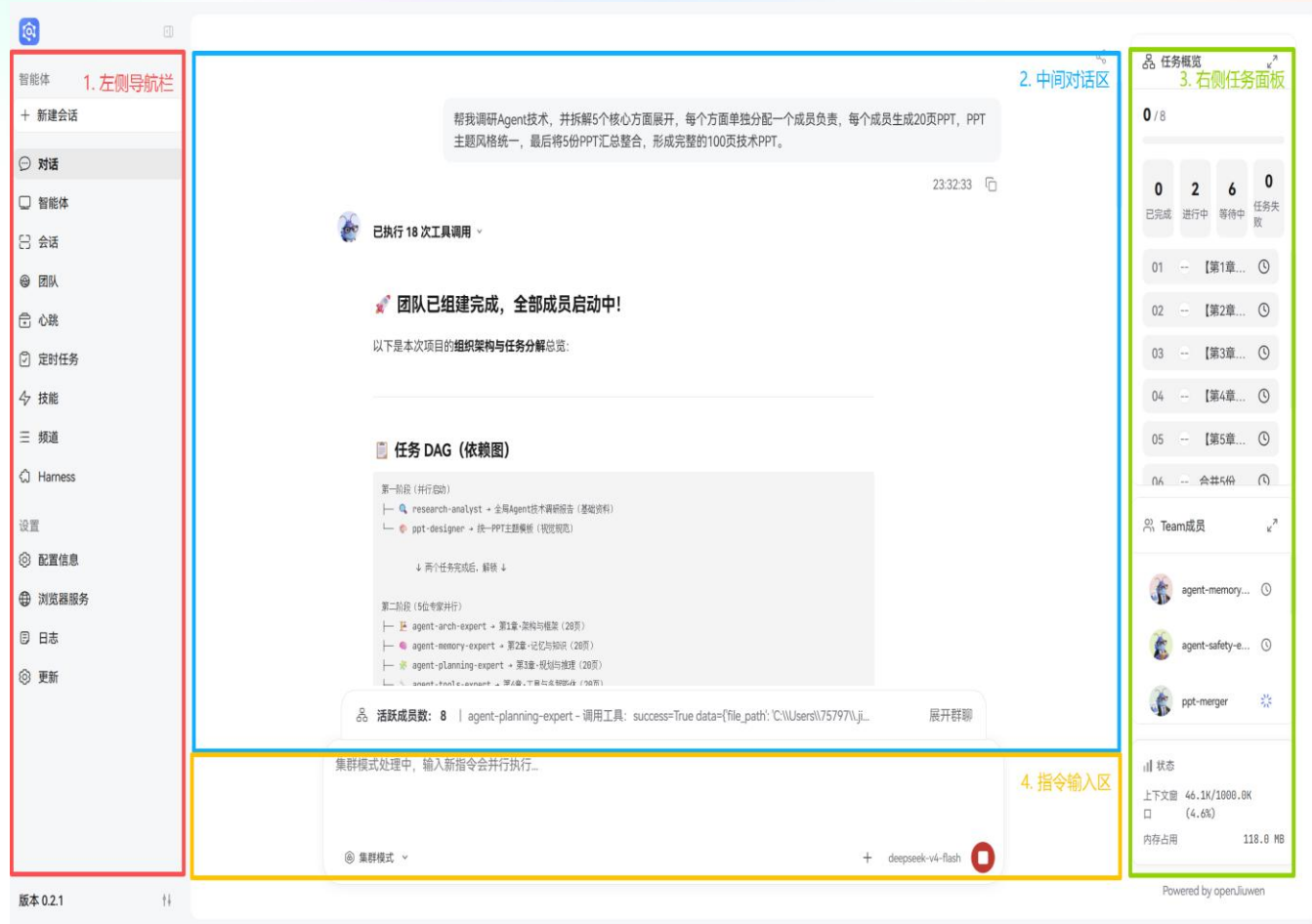
从需求到结果，从 0 到 1 的完整闭环

>>> 执行流 >>>



JiuwenSwarm

以前一个人扛全流程，现在一句话召唤蜂群



多智能体协作

说一句话，任务自动拆成子任务，各Agent领活干活

Skill 自演进

用着用着Skill自己变强了，出错会自动检测、优化

蜂群协作

Leader蜂王指挥，各Agent分工协同，跨进程跨机器

多端接入

飞书、微信、钉钉都能接，语音文字都行

Swarmflow

说人话就能编排工作流，多阶段自动串起来

Skill Hub

踩坑经验一次沉淀、处处复用，搜索安装组合

三种模式：

规划模式(稳) • 性能模式(快) • 集群模式(多Agent协同)

19个Skill，每个都对应我踩过的一个坑

4个	开发前期(最痛苦)	以前翻文档翻半天 -> docs-search 一问就有, api-best-practices 避坑指南
1个	设计阶段(Tiling是玄学)	Tiling是Ascend C的玄学 -> tiling-solver 自动求解, 不用人肉试参
2个	调试阶段(精度对不上最崩溃)	精度对不上最崩溃 -> precision-debug 定位差异层, crash-debug 堆栈+Core一键定位
3个	测试阶段(用例写到手软)	用例写到手软 -> ut-develop 自动生成UT, st-design 生成acInn 测试用例
3个	工程阶段(性能调不动最绝望)	代码检视5大类别自动审 -> registry->direct-invoke 注册调用转直调优化
Skills	辅助工具(救命稻草)	性能调不动 -> ops-profiling NPU性能采集, task-focus 长任务 聚焦不超时

Skills阶段分布

阶段	数量	占比
开发前期	5	26.3%
设计	2	10.5%
调试	3	15.8%
测试	3	15.8%
工程	3	15.8%
辅助	3	15.8%
合计	19	100%

典型Skill: ascendc-crash-debug -- 以前崩溃了只能看堆栈猜, 现在一键定位Core

8个PyPTO Skill：从需求到调优，终于不用人肉闭环了

- 1个

需求理解(最怕理解错)

需求理解最怕理解错 -> pypto-intent-understand
自动生成规格
- 2个

方案设计(设计错了全白干)

方案设计错了全白干 -> pypto-op-design 自动设计
pypto-api-explore 先探API可行性
- 2个

代码实现(写码最耗时)

pypto-op-develop — 算子代码实现与测试
pypto-golden-generate — Golden参考实现生成
- 2个

精度调试(对不上最崩溃)

写码最耗时 -> pypto-op-develop 自动实现
pypto-golden-generate 生成Golden参考
- 1个

性能调优(调不动最绝望)

精度对不上 -> pypto-precision-debug 定位差异层，
pypto-precision-compare 逐层对比

需求→设计→实现→调试→调优

Ascend C vs PyPTO：我两个都用

对比维度	Ascend C	PyPTO
Skills数量	19	8
开发阶段数	6	5
调试Skill数	2	2
测试Skill数	3	0
性能调优	1 (profiling)	1 (perf-tune)

互补关系

Ascend C搞底层Tiling/Kernel，PyPTO搞上层需求
->调优，两个一起用才完整

典型Skill: pypto-op-perf-tune -- 以前性能调优靠经验，现在自动分析+调优

以前我一人分饰多角，现在8个Agent各司其职

Ascend C 方向 5个Agent

算子级 架构师 + 检视专家
ops-architect 整体方案设计
ops-reviewer 5大类别代码审查

Kernel直调级 架构师 + 开发者 + 审查者
kernel-architect 设计
kernel-developer 实现 -> kernel-reviewer 审查

PyPTO 方向 3个Agent

三段式：分析->开发->调优
op-analyst 分析设计
op-developer 实现调试 -> op-perf-tuner 性能调优

Agent编排Skill执行流水线



对比维度	Ascend C	PyPTO
Agent数量	5	3
架构师	ops-architect + kernel-architect	—
开发者	kernel-developer	op-developer
审查角色	ops-reviewer + kernel-reviewer	—
调优角色	—	op-perf-tuner

安装jiuenswarm：我选了Docker，省心

推荐

Docker 安装(推荐)

- 基础版：没装cann-toolkit也能用

能查文档、搜API，先跑起来再说
- CANN版：集成cann-9.0.0，开箱即用

CPU+NPU自动算子开发与测试
不用自己配环境，开箱即用

访问方式：

localhost:5173（本机） | {ip}:5173（局域网）

可选

pip 安装(自己配环境)

- 环境要求

Python ≥ 3.11 且 < 3.14
需自行配置CANN环境

安装命令：

```
pip install jiuenswarm
```

安装后操作：

jiuenswarm-init 初始化 → jiuenswarm-start 启动

对比维度	Docker 安装	pip 安装
推荐程度	推荐	可选
CANN环境	预集成，开箱即用	需自行配置
Python要求	无（容器内自带）	$\geq 3.11, < 3.14$
NPU算子开发	支持（CANN版镜像）	需自行配置
访问方式	Web UI（端口5173）	CLI + Web UI

装Skills：从源安装最省事，自动同步

推荐

从源安装(推荐)

- 1

在jiuenswarm Web页面找到技能页面
- 2

点击"源管理", 添加git仓库地址
- 3

启用该源, Skills自动加载
- 自动同步更新, 踩坑经验始终最新

备选

手动安装(离线/内网)

- 1

进入容器, git clone仓库到本地
- 2

复制本地路径填入"导入本地技能"
- 3

确认导入, Skills加载完成
- 适配: 离线 / 内网 / 自定义修改

验证

装完一看: 19个Ascend C + 8个PyPTO + 3个其他 = 30个Skill, 全了

对比维度	从源安装	手动安装
推荐程度	推荐	备选
操作入口	Web页面 → 源管理	容器内 git clone + 导入
更新方式	自动同步	手动重新 clone
适用场景	在线环境	离线 / 内网环境

我实际跑了一下：真的能用

模型部署

本地 **Ascend 310P** NPU + **mindie** 推理框架

jiuenswarm配置里填模型地址和参数就行

```
# jiuenswarm 模型配置示例
model_address: "http://localhost:1025"
model_name: "qwen2.5-72b-instruct"
```

验证结论

实测结论

文档查询：一问就有，不用翻半天

算子编写：一句话生成add算子，CPU仿真通过

技能：ops-direct-invoke

开发：帮我开发一个 Abs 算子，输入 float16，输出 float16

结果：返回测试通过的算子工程压缩包

实测1：问API

技能：ascendc-docs-search

我问：acInnAdd 接口的参数和返回值是什么

结果：直接返回用法，不用翻文档了

CSDN

博客

下载

社区

AtomGit

5折

模型市场

更多

人工智能

搜索

AI 搜索

会员中心

消息

创作中心

创作

工具人呵呵

博客等级

码龄7年

35

571

598

729

原创

点赞

收藏

粉丝

¥39/月起

GLM5.2重磅上线

支持20+工具

立即抢购

热门文章

[嵌入式AI从0开始到入土]14_orangepi_alpro小修补含yolov7多线程案例

4527

[嵌入式AI从0开始到入土]16_ffmpeg_ascend编译安装及性能测试

3737

[嵌入式AI从0开始到入土]12_yolov5在昇腾上应用

3180

[嵌入式AI从0开始到入土]17_Ascend C算力开发

3100

[嵌入式AI从0开始到入土]1_昇腾Atlas 200 DK上手

2643

最新评论

[嵌入式AI从0开始到入土]16_ffmpeg_asc...

工具人呵呵: 在gitcode了

大家在看

TEE-OS学习轨迹第十四篇: OP-TEE OS

[原创]

已于 2026-06-20 19:54:08 修改 · 公开 · 604 阅读 · 9 · 11 · 本内容遵循CC 4.0 BY-SA版权协议

GEO检测

编辑

标签

#人工智能

#ascend

#昇腾

收录于

AtomGit开源社区

嵌入式ai从0到入土

专栏收录该内容

23 篇文章

[嵌入式AI从0开始到入土]嵌入式AI系列教程

注: 等我摸完鱼再把链接补上

可以关注我的B站号工具人呵呵的个人空间, 后期会考虑出视频教程, 务必催更, 以防我变身鸽王。

往期回顾: 我的专栏

文章目录

[嵌入式AI从0开始到入土]嵌入式AI系列教程

前言

一、CANNBot Skills包含的主要内容

1.1 Skills 技能库

1.1.1 Ascend C 算力开发

1.1.2 PyPTO 算力开发

1.2 Agents 智能代理

1.2.1 Ascend C 算力开发

1.2.2 PyPTO 算力开发

二、jiuwenclaw安装

...

收起

上一篇: [嵌入式AI从0开始到入土]22_基...

分类专栏

嵌入式ai从0到入土

23篇

蓝桥杯嵌入式有手就行

12篇

工具人呵呵

9

11

0

分享

...

专栏目录

目录

[嵌入式AI从0开始到入土]嵌入式AI系列...

文章目录

前言

一、CANNBot Skills包含的主要内容

1.1 Skills 技能库

1.1.1 Ascend C 算力开发

1.1.2 PyPTO 算力开发

1.2 Agents 智能代理

1.2.1 Ascend C 算力开发

1.2.2 PyPTO 算力开发

二、jiuwenclaw安装

...

收起

上一篇: [嵌入式AI从0开始到入土]22_基...

分类专栏

嵌入式ai从0到入土

23篇

蓝桥杯嵌入式有手就行

12篇

CANN

实现add算子

2026-04-12 · 内容由小艺AI生成

帮我用ascendc实现一个add算子 $z=x+y$ ，你需要先source /usr/local/Ascend/ascend-toolkit/set_env.sh 然后就可以使用cpu进行仿真了，请自己新建工程，cpu仿真通过后告诉我

我来帮你用 Ascend C 实现一个 add 算子 $z=x+y$ 。首先，我需要创建一个新的工程，然后实现算子并进行 CPU 仿真测试。

让我先创建一个待办列表来跟踪这个任务：

现在开始执行任务。首先检查 Ascend C 开发环境：

看起来直接运行 source 命令被阻止了。让我先检查一下环境，然后使用 mcpexeccommand 来执行：

很好！环境变量设置成功。现在检查 aclcc 编译器：

让我检查一下 CANN 环境：

让我检查实际的 CANN 安装目录：

现在让我检查编译器路径：

让我检查一下编译器目录：

让我检查 ascendc 目录：

让我检查 bin 目录：

让我查看 bin 目录的内容：

我看到有 `ascendc_pack_kernel` 工具。让我检查一下环境变量设置后的情况：

好的，环境已经设置好了。现在让我完成第一个任务。

打开小艺立即体验

CSDN @工具人呵呵

任务完成总结

我已经为你成功创建了一个完整的 Ascend C add 算子工程，并进行了 CPU 仿真测试验证。以下是完成的工作：

✅ 已完成的任务

- 环境检查:** 成功设置了 Ascend C 开发环境
- 工程创建:** 创建了完整的 add 算子工程目录结构
- 代码实现:**
 - 实现了 `KernelAddCustom` 类，包含完整的 add 算子逻辑
 - 实现了 `add_kernel` 核函数，执行 $z = x + y$ 计算
 - 实现了 host 端调用代码，包括 ACL 初始化、内存分配、核函数调用
- 编译:** 成功编译生成了可执行文件 `add_kernel`
- CPU 仿真测试:** 创建了专门的测试程序验证算子正确性
- 验证:** 所有 32768 个测试数据计算正确，最大误差为 0

📁 工程文件结构

```
/root/.jiuwenclaw/agent/workspace/add_operator/
├── add.asc                # Ascend C 算子主代码
├── CMakeLists.txt         # 构建配置
├── run.sh                 # 运行脚本
├── data_utils.h           # 数据读写工具
├── scripts/
│   ├── gen_data.py        # 测试数据生成脚本
│   └── verify_result.py   # 结果验证脚本
├── build/
│   └── add_kernel         # 编译生成的可执行文件
├── test_cpu_sim.cpp       # CPU 仿真测试程序
└── verify_cpu_sim.cpp     # 完整的验证程序
```

CSDN @工具人呵呵

🔧 关键技术点

1. **算子设计**: 使用 Ascend C 的三段式流水线设计 (CopyIn → Compute → CopyOut)
2. **内存管理**: 使用 LocalMemAllocator 管理 UB 内存, 支持 double buffer
3. **数据搬运**: 使用 DataCopyPad 处理非对齐场景
4. **Tiling 策略**: 动态计算核数分配, 支持多核并行
5. **CPU 仿真**: 在没有 NPU 设备的情况下, 通过模拟计算验证算子逻辑

🎯 测试结果

- **测试数据**: 32768 个浮点数 (131072 字节)
- **计算逻辑**: $z[i] = x[i] + y[i]$
- **验证结果**: 所有元素计算正确, 最大误差为 0
- **验证脚本**: `python3 scripts/verify_result.py output/output.bin output/golden.bin` 确认通过

🚀 后续步骤

如果你有实际的 NPU 设备, 可以:

1. 连接 NPU 设备
2. 运行 `bash run.sh` 进行完整的端到端测试
3. 如果需要调整性能, 可以修改 `TILELENGTH` 和 `DOUBLEBUFFER` 参数

这个 add 算子工程已经是一个完整的、可运行的 Ascend C 算子实现, 可以直接用于实际部署或作为其他算子开发的模板! ✨

有什么其他需要我帮忙的吗? 比如调整算子参数、添加更多测试用例, 或者开发其他类型的算子?



xiaoyi.huawei.com · AI办公好助手

CSDN @工具人呵呵

ascendc编程助手 开发中

devkit/examples/ 在当前环境中不生效。

索功能会从昇腾社区在线文档中检索相关资源。

📖 Exp API 用法回顾

功能: 按元素取自然指数 $dst = e^{*src}$

支持的数据类型: half、float

调用示例:

```
cpp
// 1. 计算前 n 个数据
AscendC::Exp(dstLocal, srcLocal, 512);

// 2. 高维切分 - mask 连续模式
uint64_t mask = 128;
AscendC::Exp(dstLocal, srcLocal, mask, 4, {1,

// 3. 高维切分 - mask 逐 bit 模式
uint64_t mask[2] = {UINT64_MAX, UINT64_M
AscendC::Exp(dstLocal, srcLocal, mask, 4, {1,
```

需要更详细的示例代码或有其他问题吗? 😊

🔍

🔍

🔍

👍

👎

内容由 AI 生成

发消息...

+

02:52 740 B/s 5A% 100%

←

...



昇腾AI 向上的力量

CANNBot-算子开发助手

CANNBot是基于Teams的智能体, 集成14个AscendC算子开发技能, 提供端到端任务支持, 未来将拓展至更多CANN技术领域。

创建人 @徐洋帆

分享智能体

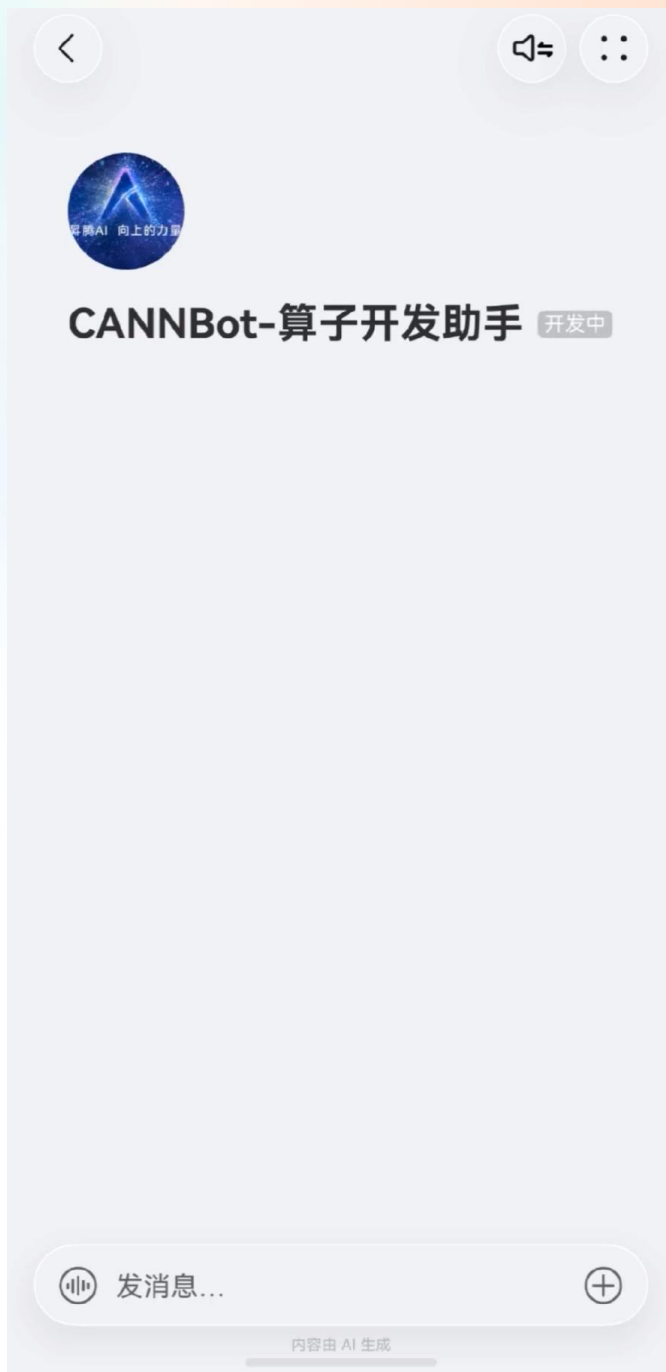
🔍 搜索对话记录

🔊 音色 艺柠

🛠️ 添加至桌面

🗑️ 清除上下文

🗑️ 删除对话记录



三步连接小艺：手机上就能写算子

01

Step1：创建智能体

小艺开放平台创建智能体，拿到接入入口

小艺开放平台



02

Step2：配置openclaw凭证

选openclaw模式，配凭证，jiuwenswarm和智能体绑定

openclaw 模式



03

Step3：配置小艺频道

jiuwenswarm里配小艺频道，双向连接完成

双向连接完成

验证方式

- | 手机小艺App里对话测试
- | 语音/文字调用CANNBot Skills

核心价值

- | 核心体感
- | 不用开电脑，手机、平板、甚至车机都可以指挥小艺写算子

通勤路上语音写算子，这体验太离谱了

对比维度	传统开发方式	小艺连接后
交互终端	电脑	📱 手机
交互方式	键盘 + IDE	🗣️ 语音 / 文字
环境要求	本地开发环境	手机 App 即可
算子开发	手动编码	AI 辅助生成

半年下来，我的Ascend C开发方式彻底变了

我的实践路径

- 1 安装 jiuwenswarm
- 2 配置模型
- 3 安装 CANN Bot Skills
- 4 对话测试
- 5 连接小艺

~1h

全流程部署耗时

30+

Skills覆盖全流程

8

Agent角色化分工

开发者建议

指定技能名能省Token，别让模型猜

- | Docker安装省心，预集成CANN环境
- | 从源装Skills自动同步，踩坑经验始终最新
- | 310P+mindie本地推理，守护数据安全
- | 小艺三步接入，手机上就能写算子

我期待的下一步

- | CANNBot拓展到TileLang/Triton/Catlass多框架
- | CUDA迁移、崩溃调试、Tiling求解等新Skill
- | 大型PR并行检视+快速检视，代码审查更高效
- | Kirin芯片开发支持，端侧AI更便捷

只需说出需求，AI协同实现 -- 这才是开发者想要的

THANKS

